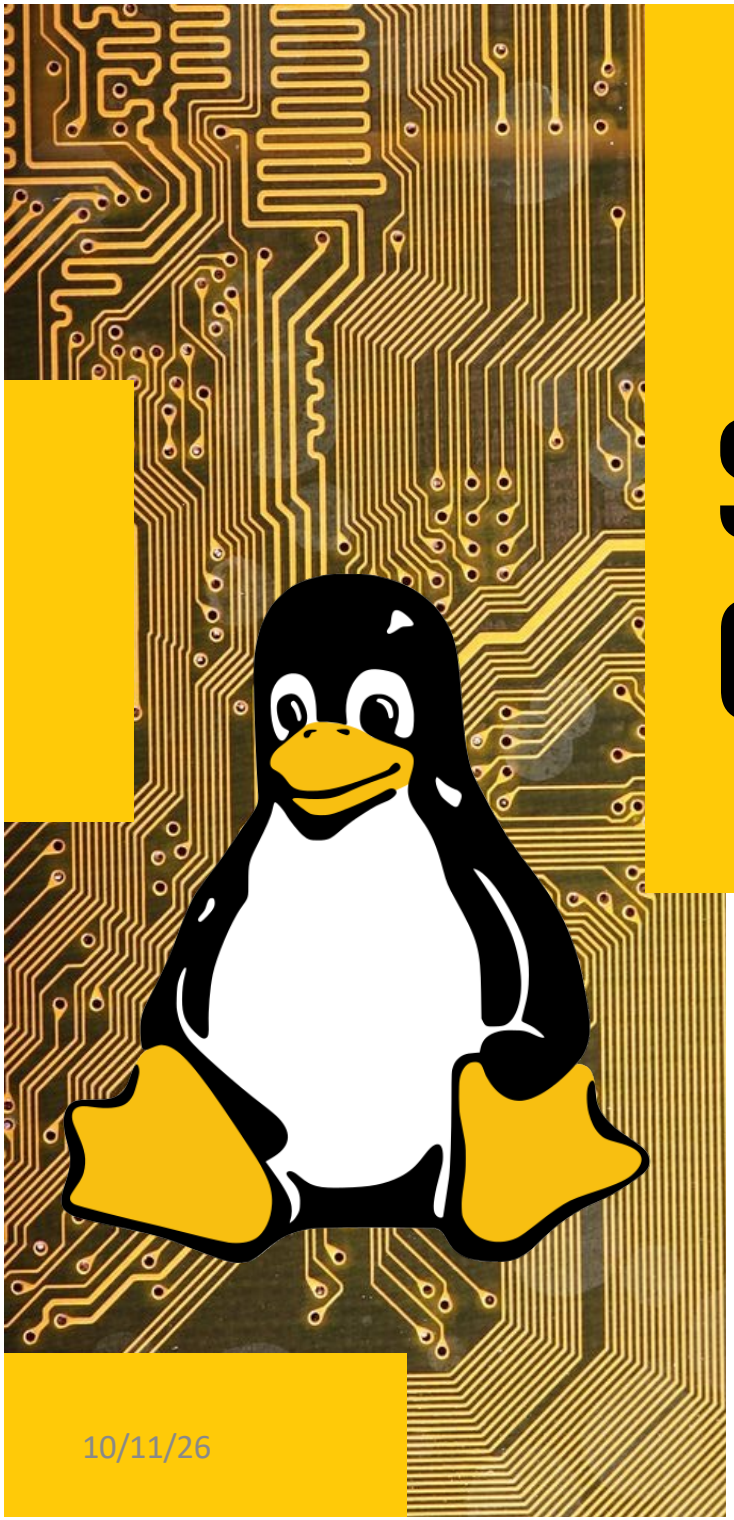


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

Synchronisation I

Semaphore Patterns

INSTRUCTOR: Jalolliddin Yusupov

What Is Synchronisation?

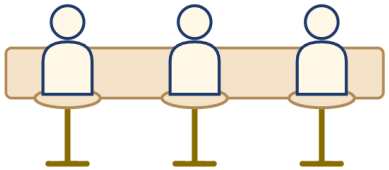
Synchronisation is the set of rules that decide **who may do what, and when**, when several processes share something. With them, the result of the program no longer depends on the order the scheduler happens to choose.

What the OS gives, what you give

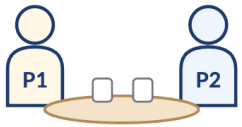
The OS supplies the **mechanism**: an atomic counter, sleep, wake exactly one. You supply the **pattern** — which process waits, which posts, and in what order. *The pattern is this week's subject.*
Synchronisation = make the answer independent of the interleaving.

Week 4 gave you one counter with two operations. This week the same counter learns five jobs — enough to run a whole café.

THE CAFÉ OF THE WEEK



three stools: a limit



two friends: a rendezvous



a tour group: a barrier



the pass shelf: a bounded buffer



Reading

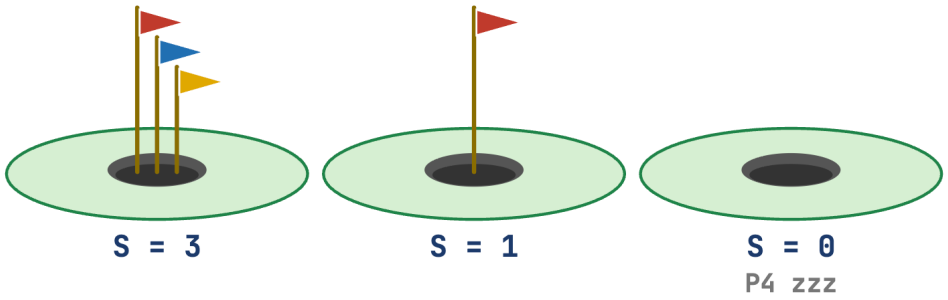
Allen B. Downey, *The Little Book of Semaphores*, ch. 1–4 (free online) · Silberschatz §6.6 · OSTEP ch. 31.

Already yours, from Week 4

`wait` and `post`, the golf flags, the trace table, `shared_mem.h`, `sem_init(&s, 1, value)`. Nothing else is needed this week.

What a Semaphore Is

A **counter** that processes can decrease and increase **atomically**, and that makes a process **sleep** when there is nothing left to take. That counter is the **semaphore**.



EXACTLY TWO (ATOMIC) OPERATIONS

```
S = Semaphore(3)

wait(S):  if S > 0:  S = S - 1
          else:      sleep until S > 0,
                     then S = S - 1

post(S):  S = S + 1
          if someone sleeps on S:
              wake ONE of them
```

Set the start value once; after that, only wait and post — never read the value.

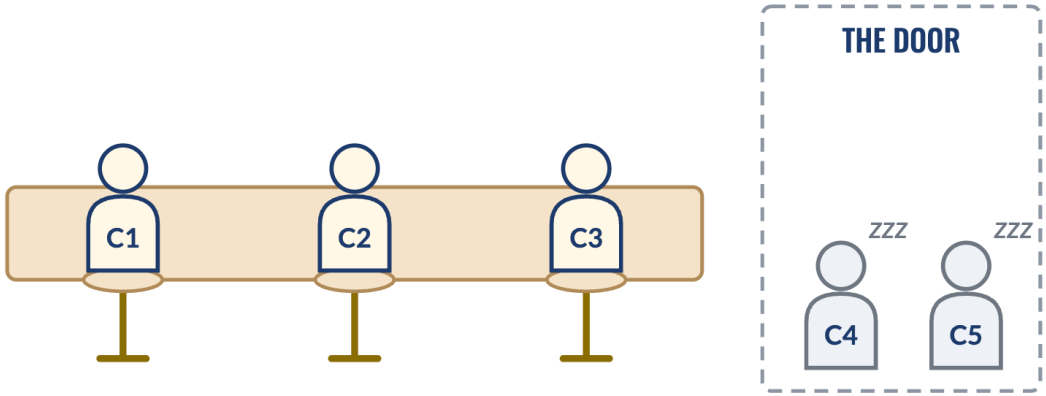
start value	the job	in the café	taught
1	**a lock**: one process inside at a time	one key to the toilet	Week 4
0	**a signal**: "A must happen before B"	"call me when the coffee is ready"	Week 4
N	**a limit**: at most N inside at once	three stools at the counter	**today**
0 and 0	**a rendezvous** (French: *a meeting point*): two processes wait for each other	two friends agree to leave together	**today**
N, 0 and 1	**a bounded buffer**: a shelf with room for N things between two kinds of process (*bounded*: it has an end, it is not infinite)	the pass shelf between the kitchen and the waiters	**today**

Two operations, every time wait and post never change.
What changes is the **start value** — and **which** process waits and which one posts.

The start value is the meaning.

Pattern 1: The Limit (Multiplex)

THE COUNTER: THREE STOOLS



stools = 0 · inside: C1, C2, C3 · asleep by the door: C4, C5

```
stools = Semaphore(3)   /* 3 free stools */

Customer:
  wait(stools)      /* sit down, or sleep
                       by the door        */
  eat
  post(stools)      /* free the stool     */
```

Multiplex

The book's name for a limit (Downey): several **P** inside at the same time, but **never more than N**.

A lock is a limit of 1

Semaphore(1) is exactly this pattern with one stool.

Who gets the free stool?

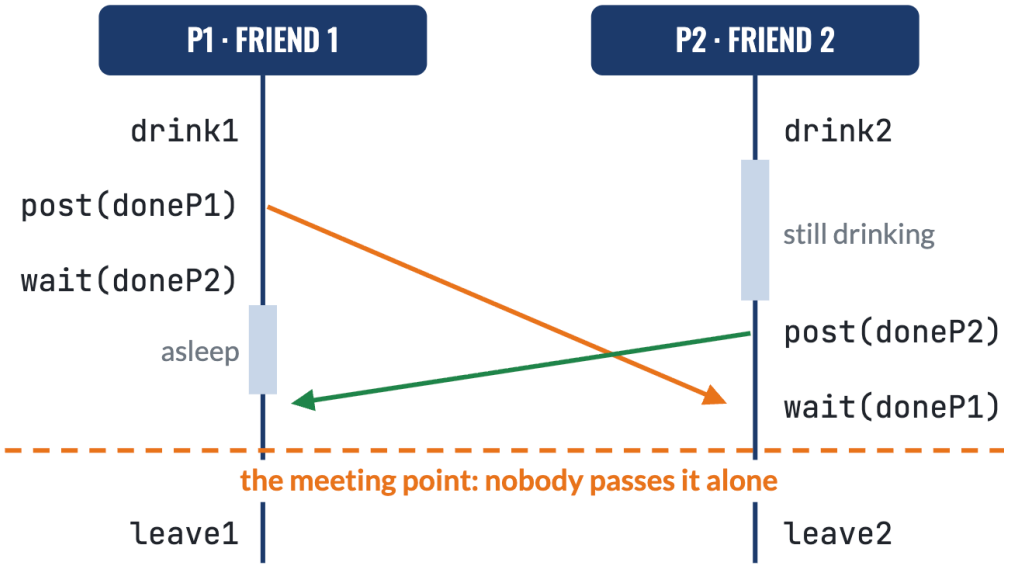
The OS: `post` hands the stool straight to **one** sleeper. You cannot choose which one — and a correct program never needs to.

The two lines are a pair

Every customer who passes `wait` must reach `post`. One missing `post` and the café has a stool fewer, for ever.

Start at N: at most N inside at once; the others sleep by the door.

Pattern 2: The Rendezvous



Two semaphores, both start at 0

One flag per friend: each friend posts **their own** and waits for the **other's**. The orange arrow is doneP1, the green one doneP2.

```
doneP1 = Semaphore(0)          doneP2 = Semaphore(0)

Process P1:                    Process P2:
  print("drink1")                print("drink2")
  post(doneP1)                   post(doneP2)
  wait(doneP2)                   wait(doneP1)
  print("leave1")                print("leave2")
```

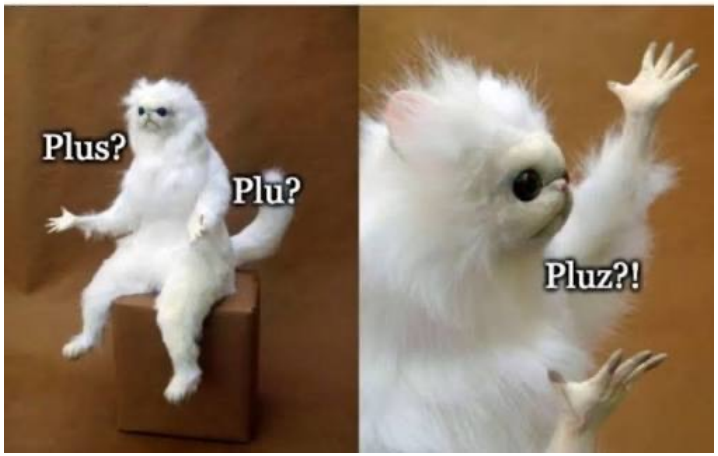
Rendezvous

French: *a meeting point*. Two processes wait for each other at one place in their code: **neither passes it until both have arrived**. In the café: both must finish their coffee before either leaves.

Why it works

`post(doneP1)` says "I have finished my coffee".
`wait(doneP2)` says "I do not leave before you have finished yours". Each one announces first, then waits.

My English-speaking brain trying to remember French pronunciation rules:



Pattern 3: The Barrier (non-reusable)

arrived = 3
the shadow counter




```
N       = 4              /* the group      */
arrived = 0              /* shadow counter */
mutex   = Semaphore(1)   /* guards arrived */
gate    = Semaphore(0)   /* closed         */

Tourist:
  wait(mutex)
  arrived = arrived + 1
  if arrived == N:       /* the last one   */
      post(gate)         /* opens the gate */
  post(mutex)
  wait(gate)             /* pass ...       */
  post(gate)             /* ... and let the
                            next one in    */
  leave
```

Barrier

N processes wait at one point until all N have arrived — then all of them go on. The tour group: nobody leaves the café until the whole group is there.

Shadow counter

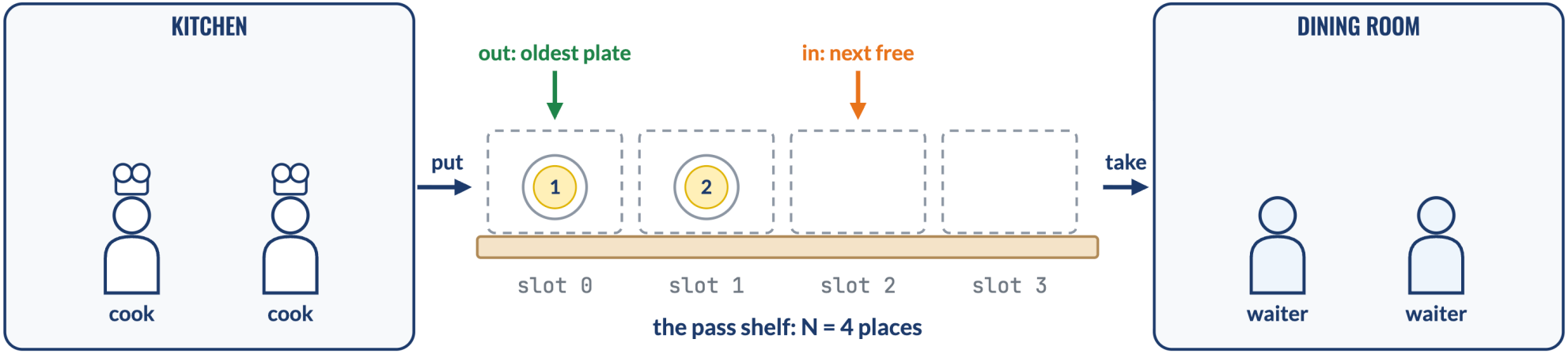
`arrived` is an **ordinary integer** that counts what you may not read from a semaphore. It is shared data, so it is guarded by `mutex` like any other.

Wait, then post at once

`wait(gate); post(gate)`
lets exactly one through and re-opens the gate for the next — like a turnstile at a metro station.

You may not read a semaphore — so keep your own count, under a lock.

Pattern 4: The Pass Shelf - Producers and Consumers



Three things must never happen

1. A plate put on a **full** shelf: it lands on top of another plate, and that one is lost.
2. A waiter taking from an **empty** shelf: he serves nothing.
3. Two waiters taking the **same** plate: one table gets it twice, another never — Week 4's lost update, with plates.

Three names

Producer — a process that makes things (the cook). **Consumer** — a process that uses them (the waiter). **Bounded buffer** — the place between them; *bounded*: it has N slots, not infinitely many.

You have used one already

A **pipe** (Week 3) is a shelf of 64 KiB of bytes: the writer sleeps when it is full, the reader when it is empty — the OS does it for you. The **listen queue** (Week 5) is a shelf of waiting callers.

Attempt 1: One Semaphore

```
N      = 2              /* places on the shelf */
plates = Semaphore(0)   /* plates on the shelf */

Cook:                          Waiter:
  loop {                         loop {
    cook a plate                   wait(plates)
    put it in the next place       take the oldest plate
    post(plates)                   serve it
  }                              }
```

Counting plates is not enough: somebody must count the free places too.

Exercise · what goes wrong?

(a) The shelf has **N = 2** places. The cook is fast, the waiter is slow. Follow plates 1, 2 and 3: where does plate 3 go, and what happens to plate 1?

(b) Now **two waiters** share the shelf. Find an order of steps in which one plate is served twice.

Hint

`plates` counts the plates that are waiting. Who counts the **free places**? And who stops two waiters from reading the same place?

Which of the three must-nevers?

Your answer to (a) and to (b) should each break one of them: a plate lost on a full shelf, an empty shelf, the same plate twice.

Attempt 2: Count Places and Plates

What went wrong with one semaphore

(a) Nothing stops the cook: plate 3 goes into place 0, on top of plate 1, which nobody served — a plate lost. **(b)** Two waiters read the same place before either moves on: one plate served twice.

```
N     = 2
empty = Semaphore(N)   /* free places */
full  = Semaphore(0)   /* plates      */

Cook:                  Waiter:
  loop {                 loop {
    cook a plate           wait(full)
    wait(empty)            take a plate
    put the plate          post(empty)
    post(full)             serve it
  }                      }
```

Two counters, one shelf

`empty` counts free places (the cook waits on it); `full` counts plates (the waiter waits on it). Each one posts the other's.

empty + full = N, whenever nobody is in the middle of a step.

Exercise · trace it: one cook, one waiter

step	who	operation	empty	full	sleeping
0		(start)	2	0	—
1	Cook	wait(empty)			
2	Cook	post(full)			
3	Cook	wait(empty)			
4	Cook	post(full)			
5	Cook	wait(empty)			
6	Waiter	wait(full)			
7	Waiter	post(empty)			
8	Cook	post(full)			
9	Waiter	wait(full)			
10	Waiter	post(empty)			

(a) Fill in **empty**, **full** and **sleeping** after every step.
(b) In which rows is `empty + full` not 2? Who is in the middle of a step there?

Attempt 3: Add the Key (the Trio)

Wait for a place first, then for the key; post in the mirror order.

ANSWER — ATTEMPT 2, TRACED (N = 2)

step	who	operation	empty	full	sleeping
0		(start)	2	0	—
1	Cook	wait(empty)	1	0	—
2	Cook	post(full)	1	1	—
3	Cook	wait(empty)	0	1	—
4	Cook	post(full)	0	2	—
5	Cook	wait(empty)	0	2	Cook
6	Waiter	wait(full)	0	1	Cook
7	Waiter	post(empty)	0	1	Cook wakes
8	Cook	post(full)	0	2	—
9	Waiter	wait(full)	0	1	—
10	Waiter	post(empty)	1	1	—

(b) Rows 1, 3, 6, 7, 9: somebody holds a place or a plate that is not counted yet — in the middle of a step.

The order of the lines

Wait for a place first, **then** take the key. **Post** in the mirror order: give the key back, then announce the plate.

THE FINAL VERSION: THREE SEMAPHORES

```
empty = Semaphore(N)   /* free places */
full  = Semaphore(0)   /* plates      */
mutex = Semaphore(1)   /* the key     */

Cook:                  Waiter:
  loop {                 loop {
    cook a plate           wait(full)
    wait(empty)            wait(mutex)
    wait(mutex)            take a plate
    put the plate          post(mutex)
    post(mutex)            post(empty)
    post(full)             serve it
  }                      }
```

Why a key?

Two cooks both read the same in and put two plates into one place — Week 4's lost update. mutex makes **put** and **take** one at a time.

Exercise: Many Cooks, Many Waiters

Exercise · one key or two?

The café grows: **3 cooks** and **2 waiters** share one shelf, all running the trio.

(a) Is one `mutex` still enough?

(b) Could the cooks and the waiters have **separate** keys? Would that be correct — and why would anyone want it?

```
cookKey   = Semaphore(1)   /* guards in  */
waiterKey = Semaphore(1)   /* guards out */
```

```
empty = Semaphore(N)   /* free places */
full  = Semaphore(0)   /* plates      */
mutex = Semaphore(1)   /* the key     */

Cook:                          Waiter:
  loop {                         loop {
    cook a plate                   wait(full)
    wait(empty)                    wait(mutex)
    wait(mutex)                    take a plate
    put the plate                  post(mutex)
    post(mutex)                    post(empty)
    post(full)                     serve it
  }                              }
```



ANSWER

(a) Yes, one key is enough

Every touch of the shelf — `in`, `out` and the places — happens under the one key, whoever you are. More cooks only means a longer queue at the key.



(b) Two keys: also correct, and faster

Cooks touch only `in`, waiters only `out`, and `empty` / `full` already keep them on different places. With `cookKey` and `waiterKey` a cook and a waiter can work at the same moment.

The Classic Mistakes

mistake	what you see	fix
wrong wait order: wait(mutex) before wait(empty)	runs for a while, then the whole café is asleep (a deadlock)	wait for a place first, then for the key
a forgotten post on an error path (an early return between wait and post)	one bad order, and then one by one everybody sleeps	every way out of the critical section posts
post twice for one wait	the limit grows: four customers on three stools	exactly one post per wait
a long sleep inside the critical section	everyone queues at the key; the program is correct and slow	a lock protects data, not time: work outside it
a process exits or is killed while holding the key	everyone who needs the key sleeps for ever	no cure inside the program: never die holding it
reading the value to decide (*"if S > 0 then …"*)	works in every test, fails under load	only wait and post; if you need a number, count it yourself under a lock

The OS lesson in this table

When a process dies, the OS closes its files and frees its memory — but it does **not** put the flag back. Nobody else will either.

When a process dies holding the key, the key dies with it.

In C: The Shelf in Shared Memory

```c
/* shelf_min.c — the smallest correct shelf: one cook, one waiter, 8 slots */
#include <unistd.h>
#include <sys/wait.h>
#include <semaphore.h>
#include "shared_mem.h"                 /* also brings stdio.h and stdlib.h */
#define N 8                             /* the shelf has room for 8 plates */
#define PLATES 100000                   /* plates cooked tonight           */
struct shelf { sem_t empty, full, mutex; int in, out; int slot[N]; } *s;

int main(void) {
    s = shared_alloc(sizeof(struct shelf));        /* before fork: both see it */
    sem_init(&s->empty, 1, N);          /* 1 = shared by processes; N free slots */
    sem_init(&s->full,  1, 0);          /* no plate yet                          */
    sem_init(&s->mutex, 1, 1);          /* the key is on its hook                */
    if (fork() == 0) { cook(); exit(0); }          /* the cook   */
    if (fork() == 0) { waiter(); exit(0); }        /* the waiter */
    wait(NULL); wait(NULL); return 0;  /* the café closes when both are done */
}
```

The semaphores live in shared memory, next to the shelf they guard.

Ring

`in = (in + 1) % N` : after slot N−1 comes slot 0, like the hours on a clock. `%` is the remainder: 8 % 8 = 0.

```
$ time ./shelf_min
served 100000 plates, 100000 in order
real    0m0.210s
```

In C: The Shelf in Shared Memory

```c
void cook(void) {                       /* puts plates 1 ... PLATES */
    for (int p = 1; p ⩽ PLATES; p++) {
        sem_wait(&s→empty);             /* wait for a free slot          */
        sem_wait(&s→mutex);             /* take the key                  */
        s→slot[s→in] = p;               /* put the plate                 */
        s→in = (s→in + 1) % N;          /* ring: after slot N-1 comes 0  */
        sem_post(&s→mutex);             /* give the key back             */
        sem_post(&s→full);              /* one more plate is waiting     */
    }
}
void waiter(void) {                     /* takes PLATES plates */
    int inorder = 0;                    /* plates that came in their turn */
    for (int i = 1; i ⩽ PLATES; i++) {
        sem_wait(&s→full);              /* wait for a plate              */
        sem_wait(&s→mutex);             /* take the key                  */
        int p = s→slot[s→out];          /* take the plate                */
        s→out = (s→out + 1) % N;        /* ring: after slot N-1 comes 0  */
        sem_post(&s→mutex);             /* give the key back             */
        sem_post(&s→empty);             /* one more free slot            */
        if (p == i) inorder++;          /* the i-th plate is plate i     */
    }
    printf("served %d plates, %d in order\n", PLATES, inorder);
}
```

Line by line = the trio

`sem_wait` is `wait`, `sem_post` is `post`; `&s→empty` is the semaphore `empty` in the shared shelf. Same order, same three semaphores.

`s->`

The shelf lives in shared memory and `s` points to it, so the cook and the waiter use **the same** `in`, `out` and slots.

In order

The cook puts plates 1, 2, 3 … The waiter checks every plate it takes: the i-th plate must be plate i (its number = the count taken so far + 1). With one cook and one waiter, *100000 in order* is the proof: no plate lost, none taken twice.



Questions?